

pi0.5 训练吞吐基准：8 卡 vs 16 卡

2026-08-31 · 两台 8×A100-80G · click_bell 各训 1 epoch · pi0.5 从官方 base 起训，`action_horizon=50` · 两轮样本数均 **73,728**

1.84× 吞吐 71.0 → 130.4 samples/s 硬件真实扩展能力	91.8% 并行扩展效率 跨机仅损失 8.2%	1.51× 1 epoch 总墙钟 24:50 → 16:24
---	--	--

结论：扩展效率 91.8%，跨机通信只吃掉 8.2%，**长训按 1.84× 算**。总墙钟只有 1.51×，是因为约 290 s 的 dataloader worker 启动和约 120 s 的 XLA 编译是固定开销、不随卡数缩放，在 16 卡那轮占了总时长的 41%——这是 1-epoch 短跑的特有现象，不能外推。

01 结果

	8 卡	16 卡	倍数		8 卡	16 卡	加速
稳态每步	7.212 s	7.853 s	—	启动 (worker+权重)	~290 s	~284 s	不缩放
吞吐 samples/s	71.0	130.4	1.84×	XLA 编译+首步	140.6 s	115.0 s	不缩放
单卡吞吐	8.87	8.15	0.918	纯训练	1031.3 s	557.6 s	1.85×
扩展效率	100%	91.8%	—	1 epoch 总墙钟	24:50	16:24	1.51×

16 卡每步耗时略高是因为每步样本数翻倍 (1024 vs 512)。两 rank 循环耗时 672.6 / 665.9 s，差 1%，无掉队节点。全程 GPU 100%、410–460 W (TDP 400 W)，**瓶颈在算力，不在数据加载或通信**。按 1.84× 外推：30k 步 @ bs512 的训练，8 卡约 60 h → 16 卡约 33 h。

02 通信方式



并行策略：纯数据并行。 `fsdp_devices=1`，参数在 16 卡全复制不切分；JAX mesh (16, 1)，数据沿 batch 轴切，每卡拿全局 batch 的 1/16。每步唯一的通信是反向后一次**跨 16 卡全局 all-reduce**：pi0.5 约 3.3 B 参数 → 单步搬约 13 GB，按 95 GB/s 理论约 0.3 s，与实测 8.2% 的损失吻合。

进程模型：每机 1 进程驱动本机 8 卡（不是每卡一进程），`jax.distributed.initialize` 会合，协调者 172.31.6.28:12356。会合后 `jax.device_count()`=16、`local_device_count()`=8，mesh 自动横跨两机。

数据面不跨机。两进程各建 dataset，按 `jax.process_index()` 用 `DistributedSampler` 分片各取一半，再用 `make_array_from_process_local_data` 拼成全局数组。两机各读自己 /data 上的数据副本。

NCCL 变量	值 · 配错的后果
NCCL_IB_HCA	mlx5_1,2,3,4 不设则只走单卡，带宽剩 1/4
NCCL_IB_GID_INDEX	3 必须。index 1 是链路本地 IPv6，跨子网卡在 INIT→RTR 超时
NCCL_SOCKET_IFNAME	eth0，带外引导通道，与 RDMA 数据面分开

前提：厂商按源 IP 分流的策略路由 (`ip rule 20000–20003`) 不能动，进程须绑各自网卡源 IP，四张卡才各跑各的。这是 783 Gb/s 与 196 Gb/s 的差别。

03 配置

模型	Pi0Config(pi05=True, action_horizon=50)，约 3.3 B	两轮唯一差异	8 卡	16 卡
起始权重	官方 pi0.5 base /data/checkpoints/pi05_base/params	进程数	1	2
数据集	cobotmagic_Sim_click_bell 1000 ep / 74,000 帧 / 3 路 480×640 视频 →	每卡 batch	64	64
		每机 batch	512	512

	224×224
优化器	AdamW + cosine, warmup 50, 1e-5 → 1e-6, EMA 关
dataloader	num_workers=32 （默认 2 远不够）

两轮唯一差异	8 卡	16 卡
全局 batch	512	1024
步数	144	72
样本总数	73,728	73,728

每卡负载、每机 dataloader 负载、吃掉的样本数三者全同，墙钟可直接对比。

计时：openpi 训练循环内埋点，首步单独 block_until_ready 剥离 XLA 编译，循环结束再次 block_until_ready，故“稳态每步”已含异步派发排空时间；总墙钟取进程启停的 shell 时间戳。8 卡 14:13:47→14:38:37，16 卡 14:38:46→14:55:10。多机需给 openpi 打三处补丁：train.py 多机入口、data_loader 按进程分片、基准计时。